48. Savunmacı Programlama (Defensive Programming) Sanatı

‘Murphy Kanunları’na Göre Kod Yazmak: Yanlış Gidebilecek Her Şey Yanlış Gidecektir
Bölüm 8 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> **🎙️ NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *“Savunmacı programlama paranoyakça bir korku mudur, yoksa NASA’nın Mars araçlarını milyonlarca kilometre ötede hayatta tutan nihai mühendislik zihniyeti midir?”*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Savunmacı programlama: Girdilerin bozuk geleceğini, ağın kopacağını ve üçüncü taraf servislerin çökeceğini varsayarak kod yazmaktır. - 📌 Fail-Safe (Güvenli Başarısızlık): Sistem çöktüğünde veriyi bozmadan veya güvenlik kapısını açık bırakmadan zararsız bir duruma geçmelidir.
> - 📌 Circuit Breaker (Devre Kesici) deseni, çöken bir servise sürekli istek atıp sunucuyu kilitlemek yerine sistemi korumaya alır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Elektrik panolarındaki sigortaları (Fuses) düşünün.

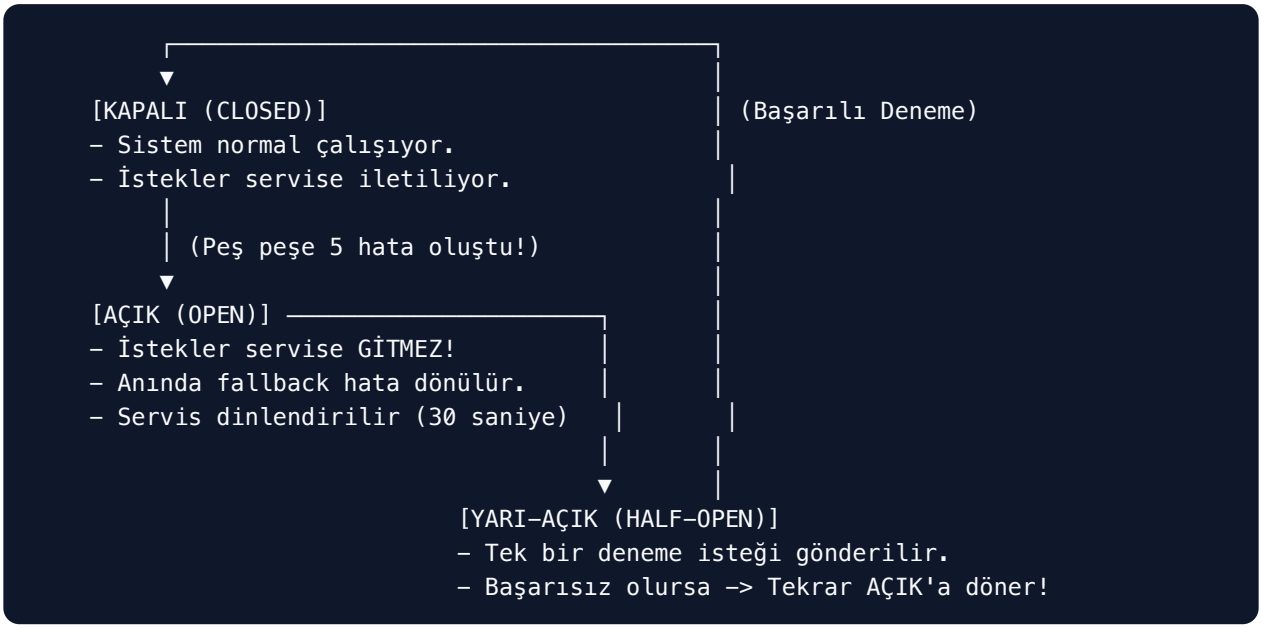
Evinize aniden 10.000 voltluk bir yıldırım düştüğünde ne olur? Sigorta anında **“ÇAT!”** diye atar ve evin elektriğini keser. Buzdolabınız veya televizyonunuz yanmaz; sadece ev karanlıkta kalır. Sigorta kendini feda ederek tüm evi korumuştur!

İşte **Savunmacı Programlama (Defensive Programming)** ve **Circuit Breaker** o elektrik sigortasıdır: Eğer ödeme yaptığınız bankanın sunucusu çöktüyse, uygulamanız kullanıcıya sonsuz yükleniyor (spinner) gösterip arkada binlerce istekle kendi sunucusunu boğmaz!

Sigortayı attırır: *“Şu an banka servislerinde geçici bir bakım var, lütfen 5 dakika sonra tekrar deneyin”* der ve sistemi güvenle ayakta tutar.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Circuit Breaker (Devre Kesici) Durum Makinesi:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Yüksek Güvenilirlikli Sistemler Mimarı
Üçüncü taraf bir döviz kuru API'sine bağımlı bir finansal servis yazıyoruz.

## GÖREV:
1. Savunmacı programlama ilkelerine uygun olarak bir Circuit Breaker deseni tasarla.
2. Servis çöktüğünde veya 2 saniye içinde yanıt vermediğinde (timeout), anında son
bilinen önbellekteki kuru dönen (Fallback) mekanizmasını kur.
3. Arka arkaya 3 hata olduğunda devreyi AÇIK (OPEN) duruma getiren tip güvenli
TypeScript kodunu yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Savunmasız Kod: Timeout yok, hata kontrolü yok, sonsuz bekleme riski!
export async function getExchangeRateNaive() {
  // Dış servis çökerse bu fonksiyon dakikalarca asılı kalır ve tüm sunucuyu
kilitler!
  const res = await fetch("https://api.currency.com/rates");
  const data = await res.json();
  return data.rates.USD;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Savunmacı Mimari: Timeout + Circuit Breaker + Fallback Kalkanı

export class ResilientCurrencyService {
  private failureCount = 0;
  private circuitOpenUntil = 0;
  private lastKnownRate = 34.50; // Fallback varsayılanı

  async getUsdRate(): Promise<{ rate: number; isFallback: boolean }> {
    const now = Date.now();

    // 1. Sigorta Kontrolü (Circuit Breaker Açık mı?)
    if (now < this.circuitOpenUntil) {
      console.warn("[Circuit Breaker] Servis açık durumda, doğrudan fallback
dönülüyor.");
      return { rate: this.lastKnownRate, isFallback: true };
    }

    try {
      // 2. Zaman Aşımı Koruması (2 Saniye Sınırı)
      const res = await fetch("https://api.currency.com/rates", {
        signal: AbortSignal.timeout(2000)
      });

      if (!res.ok) throw new Error(`HTTP Hata: ${res.status}`);

      const data = await res.json();
      this.lastKnownRate = data.rates.USD;
      this.failureCount = 0; // Başarı durumunda sayacı sıfırla

      return { rate: this.lastKnownRate, isFallback: false };
    } catch (error) {
      this.failureCount++;
      console.error(`Döviz servisi hatası (${this.failureCount}/3):`, error);

      // 3. Eşik Aşımı: 3 hata üst üste gelirse sigortayı 1 dakika attır!
      if (this.failureCount >= 3) {
        this.circuitOpenUntil = now + 60 * 1000;
        console.error("[Circuit Breaker] Sigorta attı! 1 dakika boyunca dış çağrı
yapılmayacak.");
      }

      // 4. Fail-Safe: Sistem çökmez, son bilinen güvenli kuru döner
      return { rate: this.lastKnownRate, isFallback: true };
    }
  }
}
```

💡 **Kod Analizi:** `AbortSignal.timeout(2000)` ile isteklerin sonsuza kadar asılı kalması engellenmiş, 3 ardışık hatada devre açılarak çöken servise boşuna istek atılması durdurulmuş ve son bilinen kur ile kesintisiz hizmet (graceful degradation) sağlanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Dış API isteklerine asla zaman aşımı (timeout) koymamak.
  - **Çözüm:** Her dış ağ çağrısına mutlaka 2-5 saniyelik bir `timeout` sınırı koyun.
- **Tuzak:** Hata anında uygulamanın beyaz ekran verip çökmesine izin vermek.
  - **Çözüm:** Kullanıcıya daima dostça bir hata ekranı ve alternatif bir yol (fallback) sunun.
- **Tuzak:** Savunmacı programlamayı abartıp her satıra gereksiz 10 kat kontrol eklemek.
  - **Çözüm:** Savunmayı sistem sınırlarına (API girişleri, dış servis çağrıları) yerleştirin; iç fonksiyonlarda tip güvenliğine güvenin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 48: Savunmacı Programlama - Circuit Breaker ve Fallback Tasarımı*

- 🖥️ **Slayt 1:** Murphy Kanunu: Yanlış gidebilecek her şey yanlış gidecektir.
- 🖥️ **Slayt 2:** Fail-Safe Tasarım: Çöküş anında güvenli duruma geçmek.
- 🖥️ **Slayt 3:** Circuit Breaker (Devre Kesici) durum makinesi: Closed, Open, Half-Open.
- 🖥️ **Slayt 4:** Bölüm 8 Özeti: Güvenlik, OWASP ve Savunmacı Kod Hijyeni.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir döviz servisi simülasyonu yapın. Dış servisi kasıtlı olarak kapatın. İlk 3 istekte timeout olduğunu, 4. istekte devrenin açılıp 0 milisaniyede fallback değer döndüğünü canlı konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde harici bir API çağıran fonksiyona 3 saniyelik bir `AbortSignal.timeout` ve servis çöktüğünde çalışacak bir `fallback` değeri ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım mimarisinde ‘Circuit Breaker’ (Devre Kesici) deseninin temel amacı nedir? - A) Bilgisayara yeni donanım bağlamak - B) Sürekli hata veren veya çökmüş olan bir dış servise ısrarla istek atıp sistem kaynaklarını tüketmeyi durdurmak ve sistemi güvenli bir bekleme durumuna almak - C) Sadece CSS dosyalarını birleştirmek - D) Kullanıcı şifrelerini değiştirmek > **Doğru Yanıt:** B > **Açıklama:** Circuit

Breaker, çöken bir servise yapılan çağrıları geçici olarak keserek hem o servisin toparlanmasına fırsat tanır hem de ana sistemin kilitlenmesini önler.

Soru 2: Savunmacı Programlama (Defensive Programming) felsefesine göre bir yazılımcının varsayımı ne olmalıdır? - A) Kullanıcıların her zaman doğru veri gireceği ve internetin asla kopmayacağı - B) Dış girdilerin bozuk olabileceği, ağın her an kopabileceği ve bağımlılıkların çökebileceği; sistemin bu durumlara hazırlıklı olması gerektiği - C) Kodun sadece cuma günleri test edilmesi gerektiği - D) Sadece tek bir tarayıcıyı desteklemek gerektiği > **Doğru Yanıt:** B > **Açıklama:** Savunmacı programlama ‘İyimser Kodlama’nın zıddıdır; her türlü felaket senaryosunu baştan hesaba katarak sistemi dayanıklı (resilient) kılar.